Querygraph

covers querygraph (0.1.0)

A Governed Semantic Lakehouse for AI Agents

Alexy Khrabrov

chiefscientist.org

&

Codex with ChatGPT 5

Contents

Preface

Querygraph is an AI navigator over governed enterprise data. It treats a lakehouse as more than tables: each dataset is described by Semantic Croissant, projected through CDIF, governed by RBAC and ODRL, addressed by DIDs, and audited through OpenLineage.

The implementation in this repository is intentionally practical. It can load Dataverse and CODATA data into Sail, materialize typed tables, generate Croissant and CDIF sidecars, wrap agent requests with TypeSec TypeDID envelopes, and emit OpenLineage events back into Sail. The point of this book is to make that architecture legible.

The Vision from QueryGraph.ai

The public QueryGraph.ai posts describe a larger ambition than a metadata library. They describe an AI Navigator: a system that lets agents move through data with the same precision that a navigation system gives to physical travel. The navigator does not merely retrieve documents. It finds the right semantic object, checks whether the agent is allowed to use it, records the provenance of every step, and makes the resulting answer reproducible.

The motivation begins with a critique of contemporary LLM systems. They are fluent but unstable. The same prompt can drift across time, model version, provider, and context window. For casual chat this may be charming. For science, policy, medicine, finance, infrastructure, and enterprise operations, it is a control failure. A reliable AI system needs context, provenance, and governance before it needs more tokens.

The posts make several claims that become design principles in Querygraph:

- Data must be understandable by both humans and agents.
- Ontologies must carry domain meaning, units, hierarchies, and controlled vocabularies.
- AI answers should be reproducible enough to inspect and compare over time.
- Prompts, skills, variables, datasets, and agents need identities.
- Governance must be machine-actionable and attached to the data itself.
- Graphs and vectors should work together: the graph provides identity, provenance, and relationships; vector search provides similarity and discovery.
- A responsible AI platform starts at the data layer, not at the last safety prompt before inference.

In this vision, an AI Navigator is not a chatbot. It is the semantic operating layer for agentic AI.

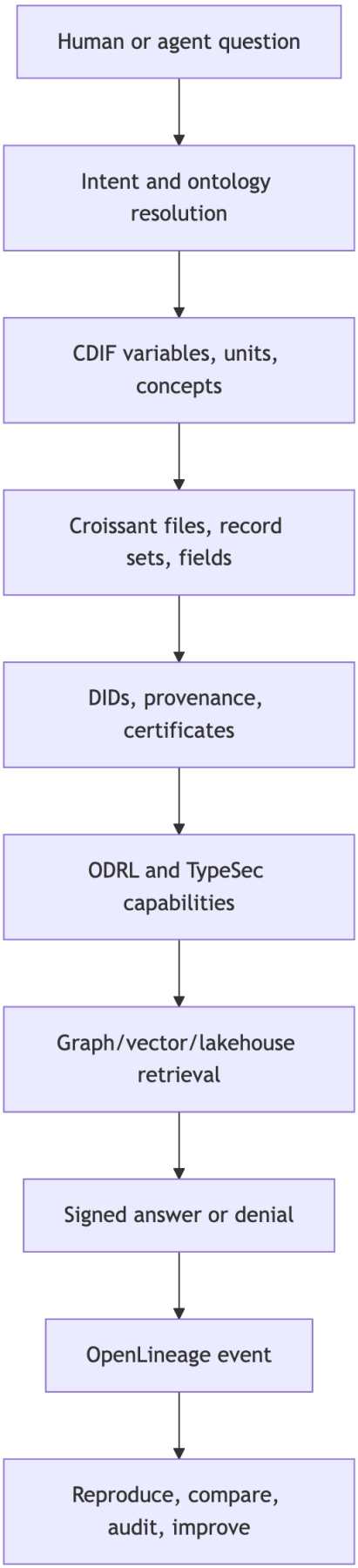

Figure 1: Diagram 1

Compression, Not Noise

One QueryGraph.ai theme is compression: a system understands something only when it can reduce noise to a portable, structured signal. Querygraph applies that idea to enterprise data. A raw table with 800 columns is not yet knowledge. A table described by Croissant, grounded in CDIF variables, linked to ontology terms, governed by ODRL, and indexed in a graph has been compressed into a form an agent can safely use.

Example: an energy survey may contain a column that looks like a number. The compressed semantic signal says what the number means, what unit it uses, which geography it belongs to, which survey instrument produced it, whether a given agent may summarize it, and what redaction rule applies before sharing.

Freezing Time Without Freezing the World

Another theme is vector stabilization and temporal ground truth. Querygraph does not claim that all knowledge is timeless. It assumes the opposite: knowledge changes, models drift, policies change, and experts disagree. The task is to record the time, source, authority, model, prompt, and data state behind an answer so later agents can compare one answer to another.

Example: a climate-health briefing generated on June 14, 2026 should be replayable against the same lakehouse manifest, Croissant/CDIF sidecars, TypeDID envelopes, OpenLineage event, and DID attestation hash. If the answer changes after a model upgrade or a dataset correction, Querygraph should show what changed.

The Data Infrastructure for Agentic AI

The QueryGraph.ai data-infrastructure post argues that responsible AI needs context, provenance, and governance. Querygraph maps those directly:

- Context comes from Semantic Croissant, CDIF, OSI, controlled vocabularies, and Grust graph relationships.
- Provenance comes from Dataverse metadata, file hashes, OpenLineage events, DIDs, and signed TypeDID envelopes.
- Governance comes from RBAC, ODRL, TypeSec capabilities, and access receipts.

Example: an agent asks for a mobility-risk prediction. Querygraph does not hand it every transportation table. It resolves the question to a mobility compartment, identifies dockless-transportation and pedestrian-injury tables, checks whether the agent may derive a summary, records the run, and returns only a signed summary.

Graph and Vector Navigation

The Palefire posts point toward graph-plus-vector navigation. Querygraph's current Rust implementation emphasizes the graph and lakehouse side, but the architecture leaves room for vector stores. The graph identifies things: datasets, variables, policies, agents, claims, prompts, and lineage events. Vectors help discover similar things: related papers, near-synonymous terms, translation candidates, or concept clusters.

Example: a user asks about "energy burden." A vector search may find related phrases such as "energy insecurity" or "access to clean cooking." The graph then decides which are official terms, which datasets contain them, what ontology defines them, and what policies govern them.

Digital Public Infrastructure

The AgStack post matters because it shows the same pattern outside a single company. Agriculture, climate, health, food security, and geospatial identity all need shared infrastructure. Querygraph's lakehouse example is enterprise shaped, but the architecture works for public infrastructure too:

- open datasets with rich metadata;
- decentralized identities for contributors and data products;
- policies that allow reuse while protecting sensitive resources;
- graph navigation across domains;
- local AI agents that can work even when central infrastructure is limited.

Example: a regional food-security agent could combine weather, crop, soil, market, and logistics data. It should know which datasets are public, which are licensed, which are embargoed, and which are local to a cooperative.

Ontology-Driven Precise AI

The strongest version of Querygraph is ontology-driven precise AI. It is not satisfied with "probably relevant chunks." It wants the exact concept, the exact variable, the exact unit, the exact source, and the exact permission.

Precision does not mean rigidity. The navigator can still use LLMs, embeddings, and agents. But those systems operate inside a semantic frame:

1. The question is mapped to ontology terms.
2. Terms are resolved to CDIF variables and OSI business concepts.
3. Variables are mapped to Croissant fields and Sail columns.
4. DIDs identify the requester, agent, service, dataset, and attestation issuer.
5. ODRL and TypeSec determine which actions are permitted.
6. Grust traverses the graph of datasets, policies, variables, agents, and lineage events.
7. Sail executes table access and stores audit records.
8. Ollama or another model receives only the governed prompt it is allowed to process.
9. OpenLineage records the run.
10. A DID attestation signs the root.

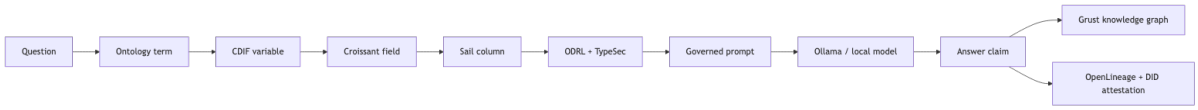

Figure 2: Diagram 2

This is the opposite of a giant ungoverned context window. It is a precise navigation path through meaning, authority, and permitted action.

Source Posts

This vision section synthesizes the QueryGraph.ai posts on CODATA, Semantic Croissant, responsible AI infrastructure, vector stabilization, compression, Palefire, and AgStack:

- https://querygraph.ai/alexy-interviews-slava-tykhonov-at-codata/
- https://querygraph.ai/moving-toward-a-verifiable-ai-future/
- https://querygraph.ai/compression-not-attention-is-all-you-need-for-ai/
- https://querygraph.ai/freezing-time-in-ai-how-vector-stabilization-captures-historical-ground-truth/
- https://querygraph.ai/building-data-infrastructure-for-agentic-ai/
- https://querygraph.ai/agstack/

- https://querygraph.ai/palefire/
- https://querygraph.ai/semantic-croissant/

The Querygraph Spine

The Querygraph spine has five responsibilities:

1. Describe data before agents touch it.
2. Decide whether an agent may use it.
3. Route work to compartmentalized agents.
4. Preserve signed summaries and access receipts.
5. Emit audit evidence into the same lakehouse substrate.

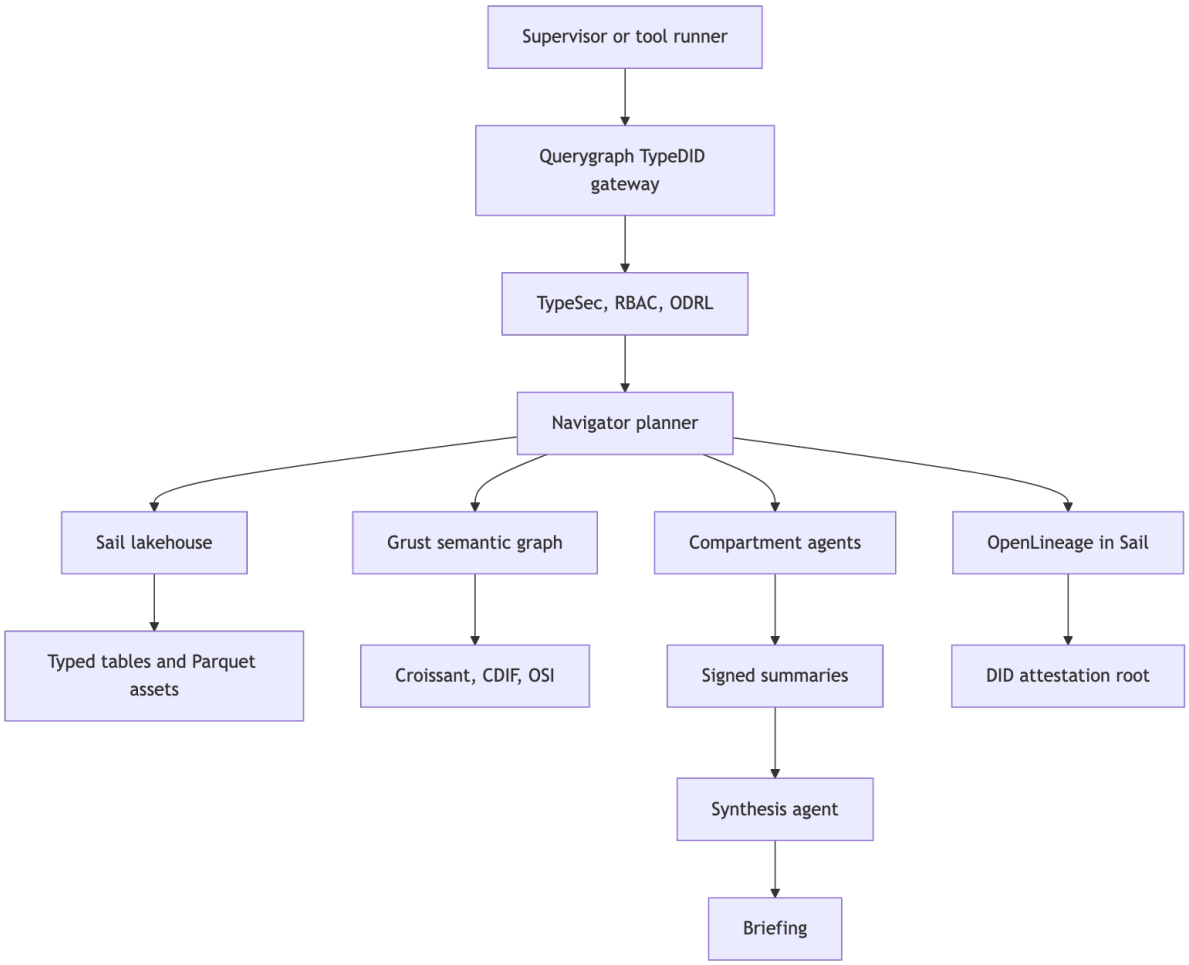

Figure 3: Diagram 3

The diagram is deliberately layered. Sail is the execution substrate. Grust is the graph substrate. TypeSec is the identity, capability, and TypeDID fabric. Semantic Croissant and CDIF make the data portable across tools and domains. OpenLineage records what happened.

Implementation Components

Every current Rust module has a role in the architecture. Querygraph is small enough that the book should name those roles explicitly:

- `croissant.rs` defines `CroissantDataset`, `FileObject`, `RecordSet`, and `Field`, then emits JSON-LD for dataset/file/field semantics.

- `cdif.rs` projects Croissant into CDIF/DCAT JSON-LD with discovery, manifest, data-description, data-access, access-rights, controlled-vocabulary, integration, universals, and provenance profiles.
- `did.rs` provides deterministic local `did:oyd` documents for demos and bundle attribution.
- `odrl.rs` models policy targets, permissions, prohibitions, actions, and the simple `allows` check used by the demos.
- `rbac.rs` maps principals to roles and roles to resource/action permissions.
- `codata.rs` calls the CODATA ODRL demo service to create URL-backed DIDs.
- `dataverse.rs` searches and fetches Dataverse metadata, parses native API responses, and turns datasets into Croissant.
- `lakehouse.rs` downloads the default corpus, normalizes CSV/TSV/XLSX/CODATA files, infers strongly typed columns, writes typed Sail tables, creates Croissant/CDIF sidecars, and verifies row counts.
- `sail.rs` stages Dataverse metadata and semantic graph nodes through Grust's Sail adapter.
- `osi.rs` loads or synthesizes Open Semantic Interchange models over datasets.
- `agent.rs` builds TypeDID request envelopes, access receipts, governed prompts, and Ollama-bound signed replies.
- `lineage.rs` constructs OpenLineage events, writes JSONL/HTTP/Sail sinks, and creates TypeSec-backed DID attestations.
- `validation.rs` validates generated Croissant, CDIF, and OpenLineage shapes.
- `qglake.rs` is the supervised multi-agent story: supervisor, specialists, restricted broker, synthesis agent, semantic projections, policies, signed summaries, and lineage.
- `navigator.rs` is the original four-layer bundle composer.
- `main.rs` wires the operator CLI commands.

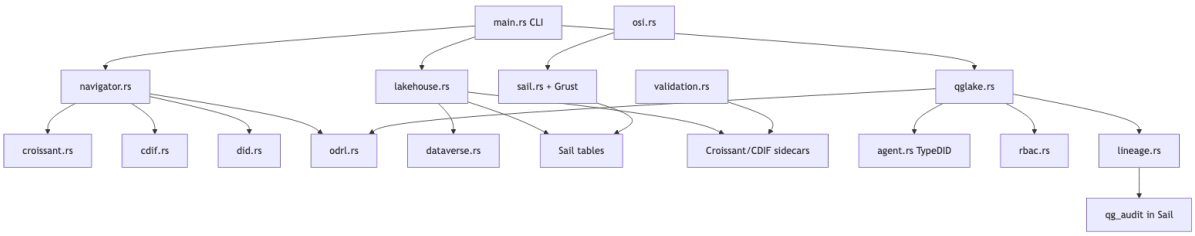

Figure 4: Diagram 4

The shape matters because each module is a boundary. Metadata generation can change without changing policy checks. The lakehouse loader can evolve without changing TypeDID. OpenLineage can be emitted to Sail while the DID ledger keeps only compact attestation roots.

Standards and Runtime Pieces

Querygraph is not one standard pretending to solve every layer. It is the coordination point for several standards and runtimes:

Piece	Role in Querygraph	Where it appears
Semantic Croissant	Dataset/file/record-set/field metadata for agent inspection.	`croissant.rs`, lakehouse sidecars, QGLake semantic catalog.
CDIF	FAIR cross-domain projection over the same dataset assets.	`cdif.rs`, `semantic/cdif.json`, access/discovery profiles.
DID	Stable identity and attribution for agents, bundles, and attestations.	`did.rs`, TypeSec DID documents, lineage issuers.
ODRL	Machine-actionable rights, permissions, prohibitions, and constraints.	`odrl.rs`, navigator policies, QGLake compartment policies.
TypeSec	Typed security fabric for DID messaging, capabilities, and verified prompts.	`agent.rs`, TypeDID envelopes, Ollama prompt binding.
TypeDID	Signed request/reply envelope for agents and tools.	`TypeDidEnvelope`, `qglake-story`, `dataverse-e2e`.
Ollama	Local model runtime reached only through a TypeSec-verified prompt path.	`call_ollama_via_typedid`, `--call-ollama`.
Grust	Property graph substrate and Sail-backed graph store.	`sail.rs`, semantic graph nodes and edges.
Sail	Lakehouse execution and audit substrate.	`qg_lakehouse`, `qg_audit`, PySpark/Spark Connect.
OSI	Business semantic model: datasets, metrics, dimensions, terms, and relationships.	`osi.rs`, Dataverse-derived or YAML-supplied model.
Dataverse	Dataset metadata and file source for the demonstration corpus.	`dataverse.rs`, `lakehouse.rs`.
CODATA	ODRL/DID anchoring demo and trusted reference-data ecosystem.	`codata.rs`, CODATA constants dataset.
OpenLineage	Operational event stream for loads, agent runs, and derivations.	`lineage.rs`, Sail audit tables, JSONL sink.

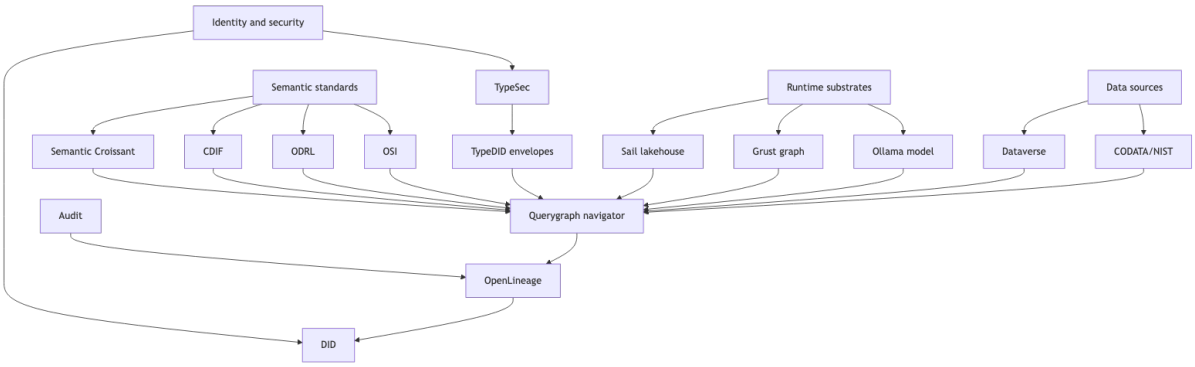

Figure 5: Diagram 5

The important design choice is that the pieces remain separable. Croissant does not decide access. ODRL does not describe file formats. TypeDID does not store the data. Grust does not replace Sail. Ollama does not receive prompts until TypeSec has verified the DID envelope and minted the needed typed capabilities.

Semantic Metadata

Querygraph uses Semantic Croissant to describe concrete files, record sets, fields, types, and semantic meanings. That is the dataset-facing contract: an agent can inspect the shape of a table before reading it.

CDIF is the publication and interoperability projection. The same dataset can be advertised through discovery, manifest, data-access, access-rights, controlled-vocabulary, data-integration, universals, and provenance concerns.

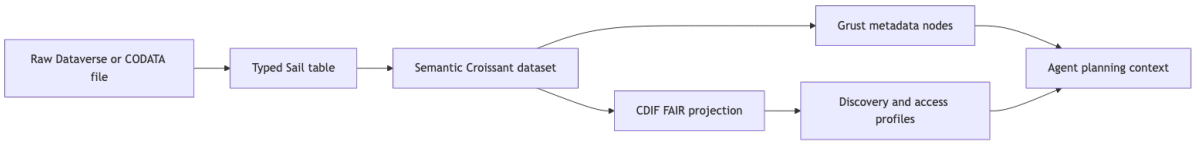

Figure 6: Diagram 6

In the local lakehouse, each loaded dataset has a `semantic/croissant.json` and `semantic/cdif.json` sidecar. The validator checks those sidecars so they do not drift away from the executable data.

Sail Lakehouse

The local demonstration loads a diverse corpus into Sail:

- government finance tables for counties, municipalities, school districts, special districts, townships, and states;
- energy access and energy insecurity survey data;
- dockless transportation, urban form, and pedestrian injury data;
- climate-health pathway data;
- CODATA constants as clean reference data;
- restricted or partially unavailable Dataverse assets that prove denials are first-class outcomes.

The result is a typed lakehouse schema, `qg_lakehouse`, with row counts verified against the load manifest. A separate audit schema, `qg_audit`, stores OpenLineage events and TypeDID attestations.

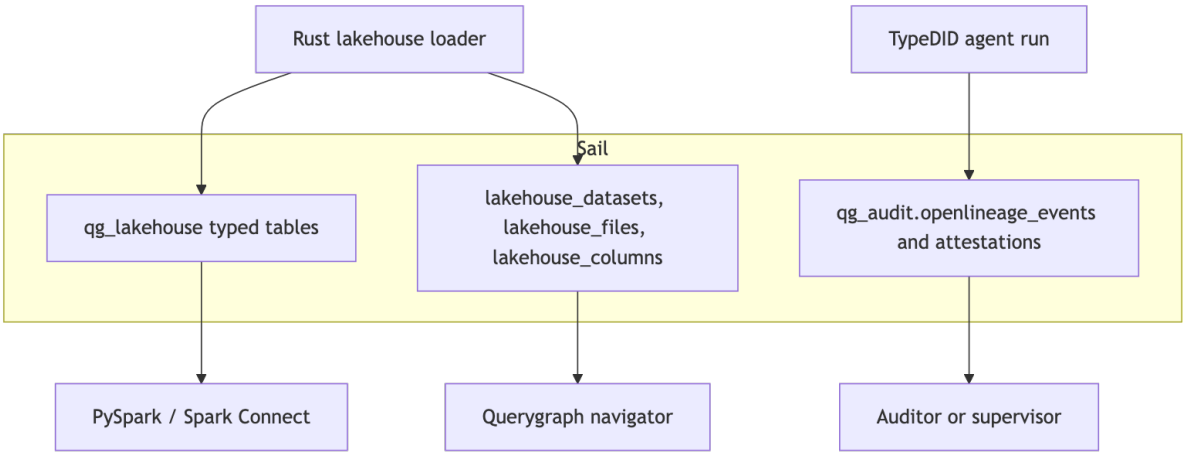

Figure 7: Diagram 7

Sail does not need to be a user interface for this to be useful. It is the local, queryable substrate for data, metadata, and audit evidence.

The Demonstration Datasets

The default corpus is intentionally broad. It is not a toy table: it mixes finance, energy, transportation, health, climate, social science, geospatial assets, and reference data.

Dataset	Category	Persistent ID or source	Typed tables	Rows	Why it is present
Government Finance Database	finance	doi:10.7910/DVN/LMS8NT	6	1,724,447	Fiscal capacity, county/ municipal/ district budgeting, and public-sector constraints.
Roadway vulnerability LiDAR DTM	geospatial	doi:10.7910/DVN/1VT6FZ	0	0	Non-tabular assets and geospatial metadata; proves the catalog can track assets that are not immediately typed tables.
ACCESS 2018 energy survey	energy	doi:10.7910/DVN/AHFINM	3	35,779	Household access to clean cooking energy and electricity.
Dockless transportation study	transportation	doi:10.7910/DVN/B2LJSB	7	479,853	Urban form, trip hotspots, mode shift, and mobility disruption.
HAALSI Baseline Survey	health	doi:10.7910/DVN/F5YHML	0	0	Restricted or inaccessible raw data; demonstrates metadata-only access and signed denial.
Global Party Survey, 2019	social science	doi:10.7910/DVN/WMGTNS	5	6,033	Institutional and political context as a social-science signal.
Connecticut pedestrian injury severity COVID-19	transportation health	doi:10.7910/DVN/IYIKEQ doi:10.7910/DVN/DMANTB table	1 2	14,645 37,982	Injury, demographic disparities in transit stops, roadway

The loaded corpus currently verifies to 28 typed tables and 2,299,541 typed rows. The row count is not the only point. The point is heterogeneity: tabular files, XLSX conversion, CODATA normalization, non-tabular assets, and restricted data all travel through one governed catalog.

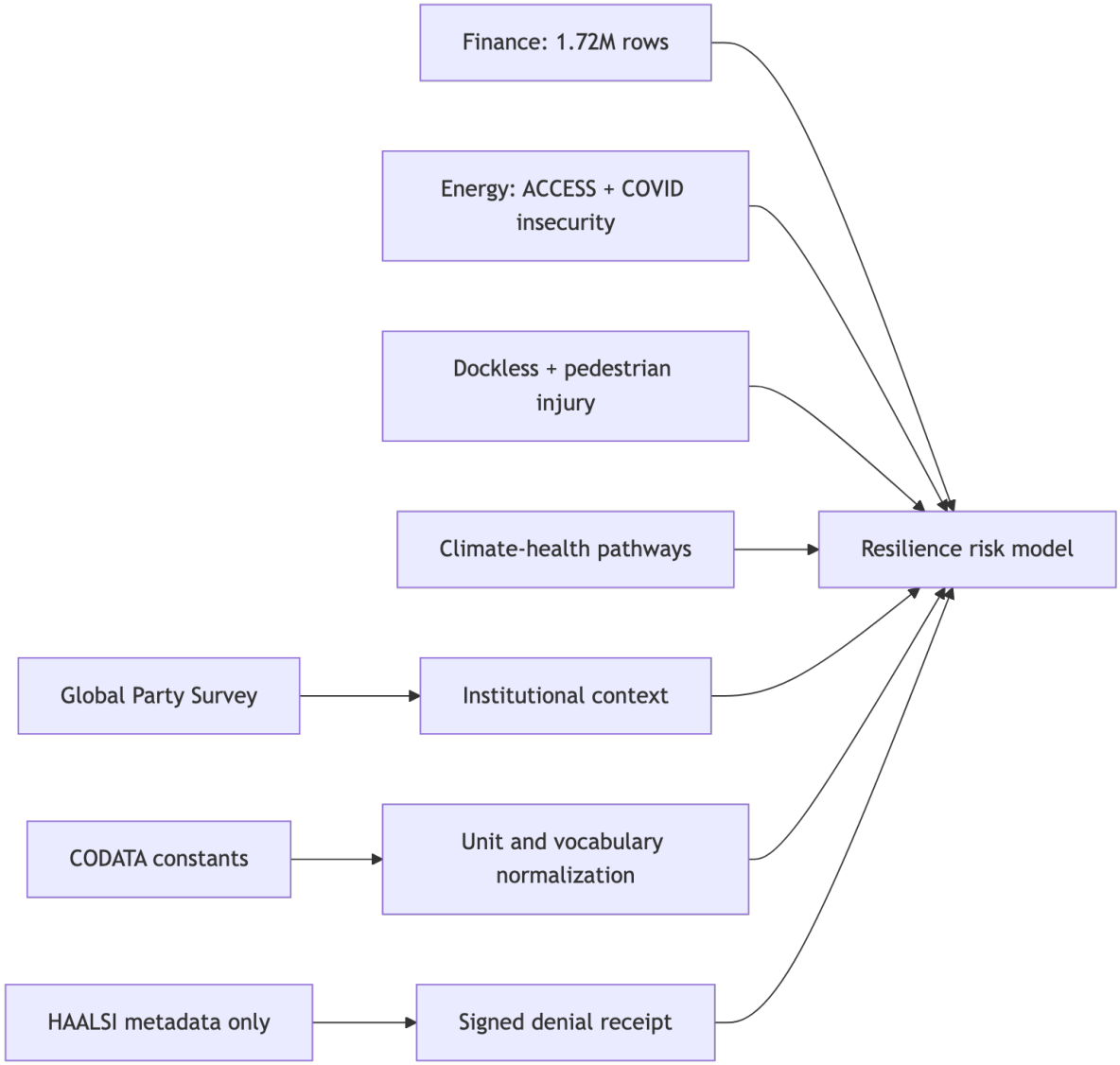

Figure 8: Diagram 8

Each dataset receives:

- raw download or source URL capture;
- normalized prepared files when parseable;
- typed Sail table materialization when tabular;
- `lakehouse_datasets`, `lakehouse_files`, and `lakehouse_columns` catalog records;
- Semantic Croissant and CDIF sidecars;
- row-count verification against the manifest.

TypeDID Agents

The QGLake story is the best compact demonstration of Querygraph's agent model. A supervisor asks:

Where do fiscal capacity, energy burden, mobility disruption, and climate-health risk overlap?

The supervisor does not get a universal raw-data token. Instead, it delegates to specialists:

- FinanceAgent reads government finance tables.
- EnergyAgent summarizes allowed energy survey fields.
- MobilityAgent summarizes transportation and injury data.
- ClimateHealthAgent summarizes climate-health pathway data.
- ReferenceAgent normalizes units and vocabularies with CODATA constants.
- RestrictedDataBroker returns a signed denial when raw restricted data is not authorized.
- SynthesisAgent aggregates signed summaries, not raw rows.

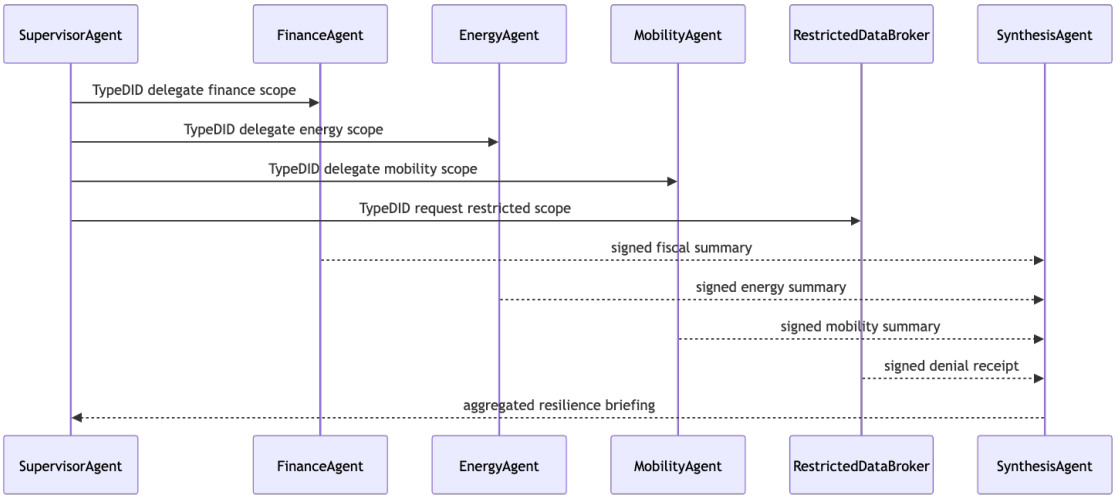

Figure 9: Diagram 9

Each specialist checks RBAC, ODRL, and TypeDID evidence. The synthesis agent receives hashes and summaries. It does not inherit raw lakehouse access.

QGLake Piece by Piece

The executable story is `cargo run -- qglake-story`. The default output is a readable briefing; `--json` returns the full machine report. The report is not a single answer. It is a trace of the whole hierarchy.

First, `SupervisorAgent` creates a TypeDID request for the resilience question. Its role is orchestration, not omniscient raw data access. It can delegate and aggregate; it does not automatically bypass compartment policy.

Second, each specialist receives a scoped TypeDID request:

Agent	Compartment	RBAC action	ODRL action	Output
FinanceAgent	`compartment:finance`	`read`	`read`	Fiscal-capacity summary.
EnergyAgent	`compartment:energy`	`summarize`	`derive`	Energy-burden summary.
MobilityAgent	`compartment:mobility`	`summarize`	`derive`	Mobility-disruption summary.
ClimateHealthAgent	`compartment:climate-health`	`summarize`	`derive`	Climate-health risk summary.
ReferenceAgent	`compartment:reference`	`normalize`	`read`	Unit and vocabulary normalization.
RestrictedDataBroker	`compartment:restricted:raw`	`read` metadata	`read` raw prohibited	Signed denial receipt.

Third, each specialist receives its own Semantic Croissant and CDIF projection. This is how the agent knows what it is allowed to reason over. FinanceAgent sees government finance tables. EnergyAgent sees energy survey scopes. RestrictedDataBroker sees restricted metadata, but the raw HAALSI-like table is not authorized.

Fourth, every response is a TypeDID response envelope. A response contains a summary, evidence string, redaction statement, and payload hash. The synthesis agent receives those signed summaries and hashes. It does not receive raw rows.

Fifth, if the run calls Ollama, the prompt is not sent as loose text. TypeSec wraps the prompt in a DID envelope, the gateway verifies the prompt subject and resource, and the policy mints typed capabilities such as `ai:infer` and `read_sensitive` for that bounded resource. The Ollama reply is signed back to the requester. In the JSON report this appears as `agentRun.ollama_typedid`.

Sixth, `SynthesisAgent` aggregates:

> Priority areas are those where weak fiscal capacity, energy burden, mobility fragility, and climate-health exposure overlap. Restricted health data contributed only a signed metadata/denial receipt, so the briefing uses approved compartment summaries rather than raw restricted rows.

Seventh, the story creates a `COMPLETE` OpenLineage event with inputs for each Sail scope and output for the resilience briefing. A DID attestation signs the event hash. In the live `dataverse-e2e` path, equivalent OpenLineage events can also be written into Sail audit tables.

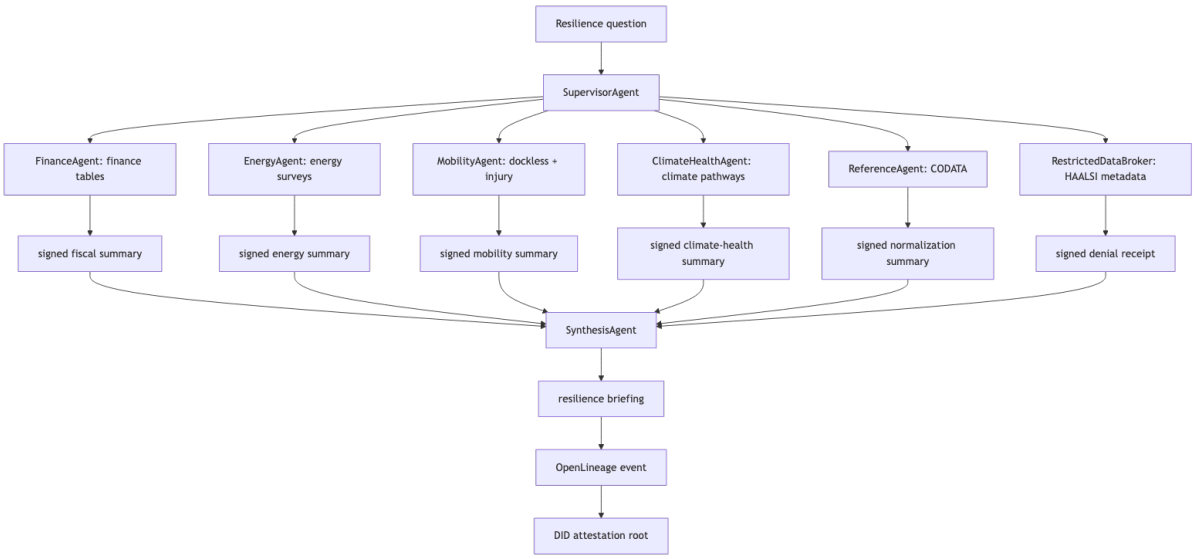

Figure 10: Diagram 10

That is the core product behavior: useful aggregation without collapsing all access boundaries.

Governance

Querygraph separates three questions that are often collapsed:

- Who is asking?
- What action is requested?
- Which semantic asset is the target?

TypeDID answers the identity and envelope question. RBAC assigns roles and permissions. ODRL expresses rights and prohibitions over the target asset. Semantic Croissant and CDIF identify the asset and its fields.

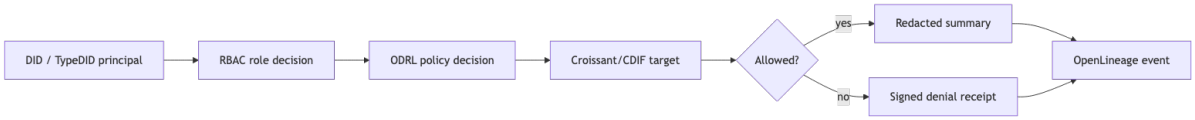

Figure 11: Diagram 11

This structure lets Querygraph say yes narrowly, no explicitly, and always leave evidence.

OpenLineage and Attestation

OpenLineage belongs in Sail itself for this local system. The event stream is queryable beside the data it describes. DID ledger entries should stay compact: store signed event hashes and Merkle roots, not every raw operational detail.

The current implementation writes:

- OpenLineage events with event hash, run id, job name, producer, and JSON body;
- TypeDID attestations with issuer, subject, Merkle root, signature, and signed payload hash.

The pairing gives operators two ways to inspect a run: query the full event in Sail, or verify a compact DID-attested root.

Operator Workflow

A normal local workflow looks like this:

```
sail spark server --port 50051
cargo run -- lakehouse-load --root .querygraph/lakehouse --schema qg_lakehouse
cargo run -- lakehouse-verify --report .querygraph/lakehouse/manifest/load-
report.json
cargo run -- lakehouse-validate --report .querygraph/lakehouse/manifest/load-
report.json
cargo run -- qglake-story
```

For the full machine-readable story:

```
cargo run -- qglake-story --json
```

For a live TypeDID, Sail, OpenLineage, and Ollama path:

```
cargo run -- dataverse-e2e \
  --live-sail \
  --sail-endpoint http://127.0.0.1:50051 \
  --openlineage-file .querygraph/openlineage/events.jsonl \
  --did-ledger-file .querygraph/did-ledger/attestations.jsonl
```

Where Querygraph Goes Next

The current codebase is the compact kernel:

- typed lakehouse loading;
- Semantic Croissant and CDIF sidecars;
- OSI model projection;
- Grust-backed semantic graph loading;
- TypeSec TypeDID agent envelopes;
- RBAC and ODRL receipts;
- OpenLineage and DID attestation.

The next product shape is a long-running `querygraphd` service with APIs for model import, semantic search, access explanation, governed planning, answer generation, and audit verification.

Querygraph's ambition is not to replace Sail, Grust, TypeSec, OSI, Croissant, CDIF, ODRL, OpenLineage, or Dataverse. It is to make them work together as one agent-safe semantic operating layer.